

# JavaScript Study Notes

*Functions, DOM, Event Handling, Arrays & Loops*

## 1. Functions

A function is a reusable block of code that performs a specific task. Instead of writing the same code again and again, it is wrapped inside a function and called whenever needed.

### 1.1 Why, When, How

- Why? — to perform a set/block of code without repeating it.
- When? — whenever the same logic needs to run more than once, or to organize code into clear steps.
- How? — using the syntax: `function functionName() { ... }`

```
function greet() {  
  console.log("Hello!");  
}  
greet(); // calling / invoking the function
```

### 1.2 Parameters, Arguments, and Variable Scope

A function can accept parameters (placeholders) and be called with arguments (the actual values passed in). Where a variable is declared decides whether it is global (accessible everywhere) or local (accessible only inside that function).

```
let a = 20; // global variable  
  
function variables(a = 20) { // a here is a parameter/argument  
  let a = 20; // local variable (shadows the global one)  
}
```

A local variable declared inside a function only exists while that function runs and cannot be accessed from outside it, while a global variable can be read or changed from anywhere in the script.

### 1.3 Types of Functions

JavaScript supports several ways of writing a function:

| Type               | Description                                                                                                      | Example                                                       |
|--------------------|------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Pure function      | Always returns the same output for the same input and does not change anything outside itself (no side effects). | <code>function add(a, b) { return a + b; }</code>             |
| Anonymous function | A function with no name, usually assigned to a variable or passed as an argument.                                | <code>const greet = function() { console.log("Hi"); };</code> |
| Arrow function     | A shorter syntax for writing functions, introduced in ES6, using <code>=&gt;</code> .                            | <code>const add = (a, b) =&gt; a + b;</code>                  |

A callback function is simply a function passed as an argument into another function, to be run later (for example, inside an event listener). A higher-order function is a function that either takes another function as an argument or returns a function.

```
// Callback example  
function processUser(name, callback) {  
  console.log("Processing " + name);  
  callback();  
}  
processUser("Ravi", function() {  
  console.log("Done!");  
});
```

## 2. Arrays

An array is a single variable that can store multiple values, in order, under one name. Each value is accessed using an index, which starts at 0.

```
let fruits = ["apple", "banana", "mango"];
console.log(fruits[0]); // apple
console.log(fruits.length); // 3
```

### 2.1 Common Array Methods

| Method          | What it does                                                      |
|-----------------|-------------------------------------------------------------------|
| push(value)     | Adds a value to the end of the array.                             |
| pop()           | Removes the last value from the array.                            |
| shift()         | Removes the first value from the array.                           |
| unshift(value)  | Adds a value to the start of the array.                           |
| indexOf(value)  | Returns the index of the first match, or -1 if not found.         |
| includes(value) | Returns true or false depending on whether the value exists.      |
| map()           | Creates a new array by running a function on every element.       |
| filter()        | Creates a new array with only the elements that pass a condition. |
| forEach()       | Runs a function once for every element (no new array returned).   |

```
let nums = [1, 2, 3, 4];
let doubled = nums.map(n => n * 2); // [2, 4, 6, 8]
let even = nums.filter(n => n % 2 === 0); // [2, 4]
```

## 3. Loops

Loops are used to repeat a block of code multiple times, instead of writing it out manually for every value or every item in a list.

### 3.1 for loop

Used when the number of repetitions is known in advance.

```
for (let i = 0; i < 5; i++) {
  console.log(i);
}
```

### 3.2 while loop

Repeats a block of code as long as a condition remains true. Useful when the number of repetitions is not known ahead of time.

```
let i = 0;
while (i < 5) {
  console.log(i);
  i++;
}
```

### 3.3 do...while loop

Similar to a while loop, but the code block runs at least once before the condition is checked.

```
let i = 0;
do {
  console.log(i);
  i++;
} while (i < 5);
```

### 3.4 for...of and for...in

- for...of — loops over the values of an array (or any iterable).
- for...in — loops over the keys/properties of an object.

```
let colors = ["red", "green", "blue"];
for (let color of colors) {
  console.log(color);
}
```

## 4. DOM (Document Object Model)

The DOM is the browser's internal, tree-shaped representation of an HTML page. Every tag becomes a node in this tree, which is what lets JavaScript read and change the page's content, structure, and styling after it has loaded.

### 4.1 The DOM Tree

Starting from the whole document, the tree branches down into elements, and further into attributes and text:

- Document → the entire webpage.
- Root element — `<html>` — the top of the visible tree.
- Element — `<head>` and `<body>` branch off the root.
- Inside `<head>`: elements like `<title>`, containing text ("My title").
- Inside `<body>`: elements like `<h1>` (text: "My header") and `<a>` (text: "My link"), which can also carry attributes such as `href`.

### 4.2 Selecting Elements

Before JavaScript can read or change an element, it must first select that element from the page:

| Method                                                | Selects                                              |
|-------------------------------------------------------|------------------------------------------------------|
| <code>document.getElementById("id")</code>            | The single element with that id.                     |
| <code>document.getElementsByClassName("class")</code> | All elements with that class (returns a collection). |
| <code>document.querySelector("selector")</code>       | The first element matching a CSS selector.           |
| <code>document.querySelectorAll("selector")</code>    | All elements matching a CSS selector.                |

```
<input type="text" id="username">
<button onclick="check()">Submit</button>

<script>
function check() {
  let username = document.getElementById("username").value;
  alert(username);
}
</script>
```

Here, `getElementById` finds the input box by its id, and `.value` reads whatever the user typed into it.

## 5. Event Handling

Events are actions that happen in the browser — a click, a key press, a page load, a form submit. Event handling means writing JavaScript that runs automatically in response to one of these actions.

### 5.1 Adding an Event Handler

There are two common ways to attach a function to an event:

- Inline, using an on-event attribute directly in the HTML, e.g. `onclick="check()"`.
- Using `addEventListener()` in JavaScript, which keeps the HTML and JS separate and allows multiple listeners on the same element.

```
<button id="myBtn">Click me</button>

<script>
document.getElementById("myBtn").addEventListener("click", function() {
    alert("Button was clicked!");
});
</script>
```

## 5.2 Common Events

| Event           | Fires when...                                   |
|-----------------|-------------------------------------------------|
| click           | An element is clicked.                          |
| change          | An input's value is changed and it loses focus. |
| keyup / keydown | A keyboard key is released / pressed.           |
| submit          | A form is submitted.                            |
| load            | The page (or an element) finishes loading.      |

## 5.3 Getting Input: `prompt()`

`prompt()` pops up a small box asking the user to type in text, and returns whatever they entered:

```
let name = prompt("Enter your name:");
console.log(name);
```

## 5.4 Conditional Statements with Events: `switch...case`

A switch statement checks a value against several possible cases — a clean alternative to a long chain of if / else if statements. It is often combined with events and `prompt()` to react differently depending on user input.

```
let num = prompt("Enter a number (1-7):");
switch (num) {
    case "1":
        alert("Sunday");
        break;
    case "2":
        alert("Monday");
        break;
    case "3":
        alert("Tuesday");
        break;
    default:
        alert("Invalid input");
}
```

Each case is checked in order; `break` stops the switch once a match is found, and `default` runs only if none of the cases match.

# 6. JSON (JavaScript Object Notation) — Quick Reference

JSON is a lightweight text format for storing and transporting data, commonly used when data is sent between a server and a webpage. It is written as key-value pairs:

```
{ "fname": "Ram", "lname": "Patil" }
```

- `JSON.stringify()` — converts a JavaScript object into a JSON string (to send it).
- `JSON.parse()` — converts a JSON string back into a JavaScript object (to use it).